Lesson 02 Demo 02
Executing Serial and Parallel Workflows Using CrewAI Agents

Objective: To demonstrate how to build intelligent workflows using CrewAI agents that execute tasks either sequentially (serial) or independently (parallel)

This demo highlights how tasks can be structured based on interdependence or concurrency.

Scenario: A product leader at a global FMCG company requires timely insights on how AI is reshaping product development, consumer engagement, and supply chain efficiency. Manually gathering and summarizing this information is time-consuming and slows product strategy decisions. To solve this, the workflow can be automated with AI agents—one dedicated to researching the latest AI trends in FMCG and another to generating executive-ready summaries. This enables the product team to make faster, data-informed decisions in a highly competitive market.

Tools required: Visual Studio Code

Prerequisites: Azure OpenAI API key

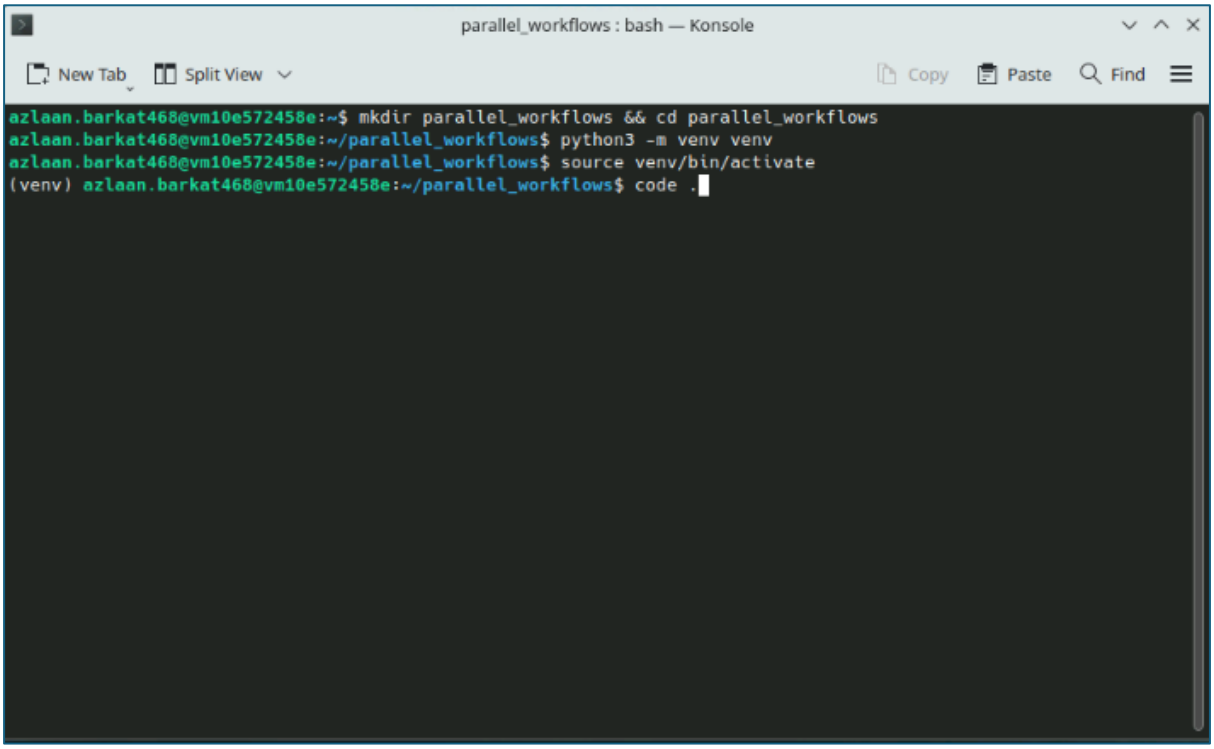
Steps to be followed:

1. Set up your local environment in Visual Studio Code
2. Configure the Tavily API Key and Azure OpenAI model
3. Define agents and tasks
4. Define and run parallel tasks

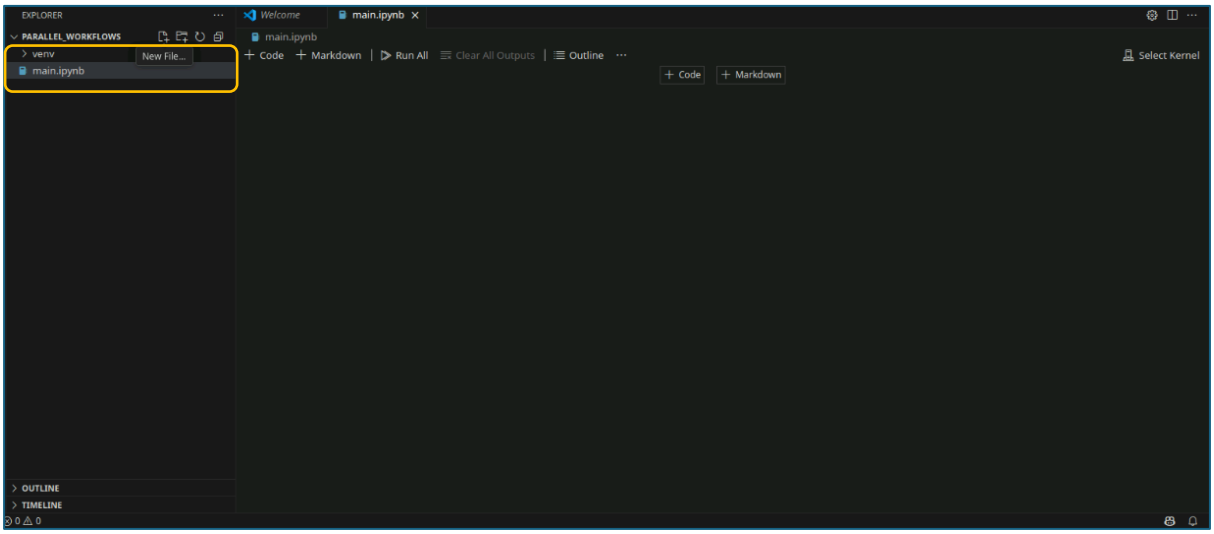
Step 1: Set up your local environment in Visual Studio Code

1.1 Open the terminal in your system and run the following commands:

mkdir parallel_workflows && cd parallel_workflows
python3 -m venv venv
source venv/bin/activate
code .



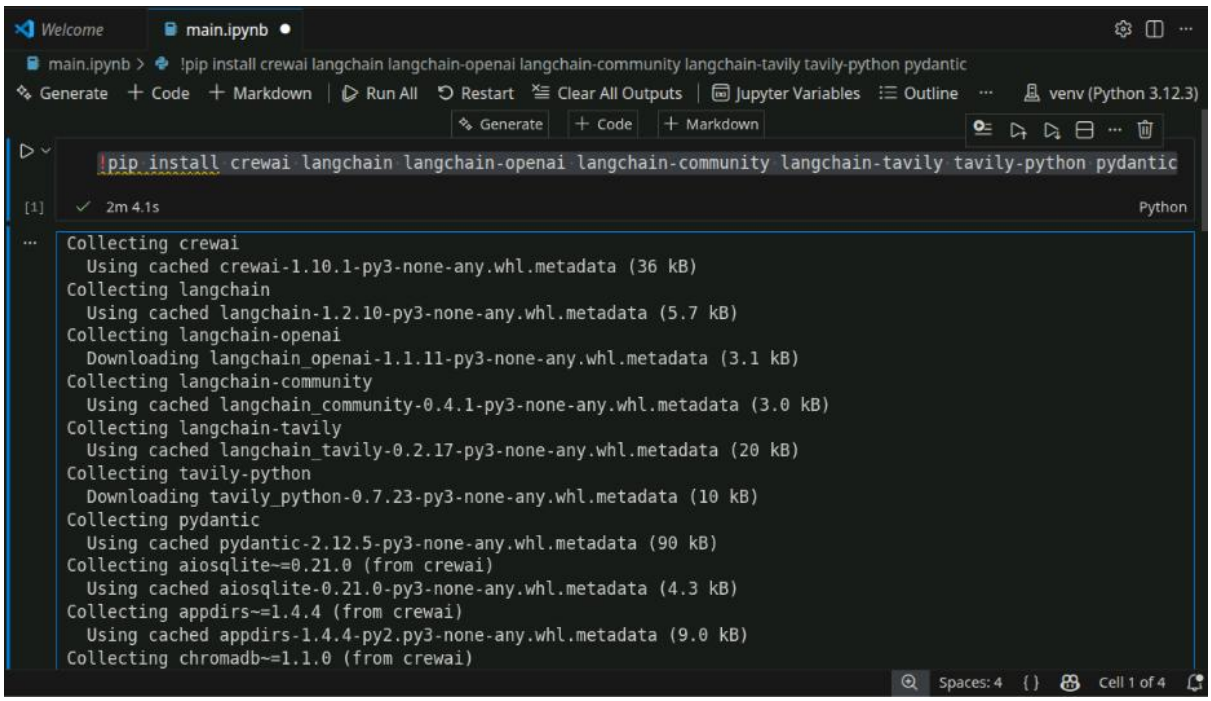
1.2 Create a new file: **main.ipynb**

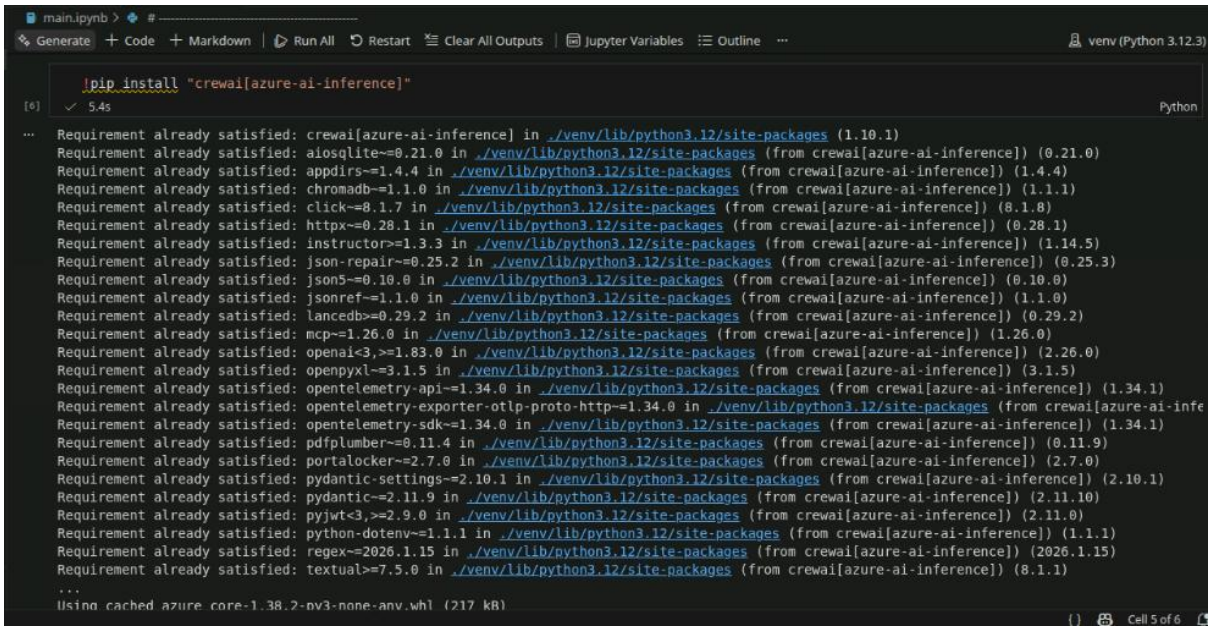


1.3 In the first 2 cells, write and run the following commands to install the required packages:

!pip install crewai langchain langchain-openai langchain-community langchain-tavily tavily-python pydantic

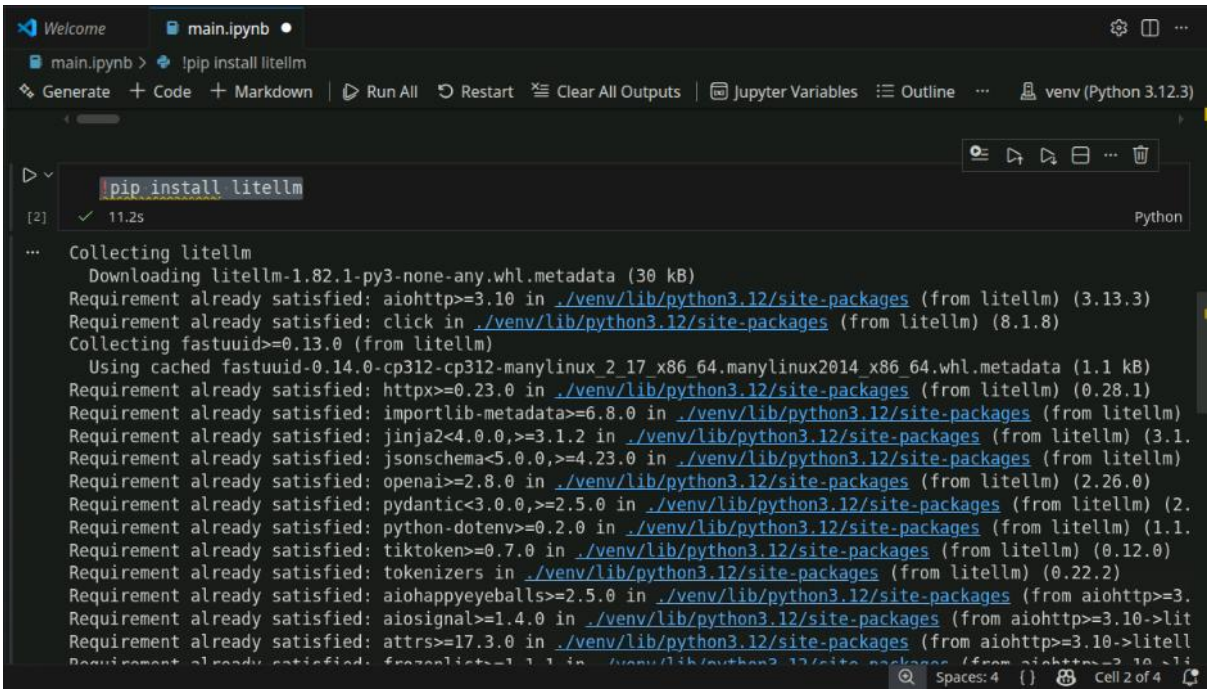
!pip install "crewai[azure-ai-inference]"





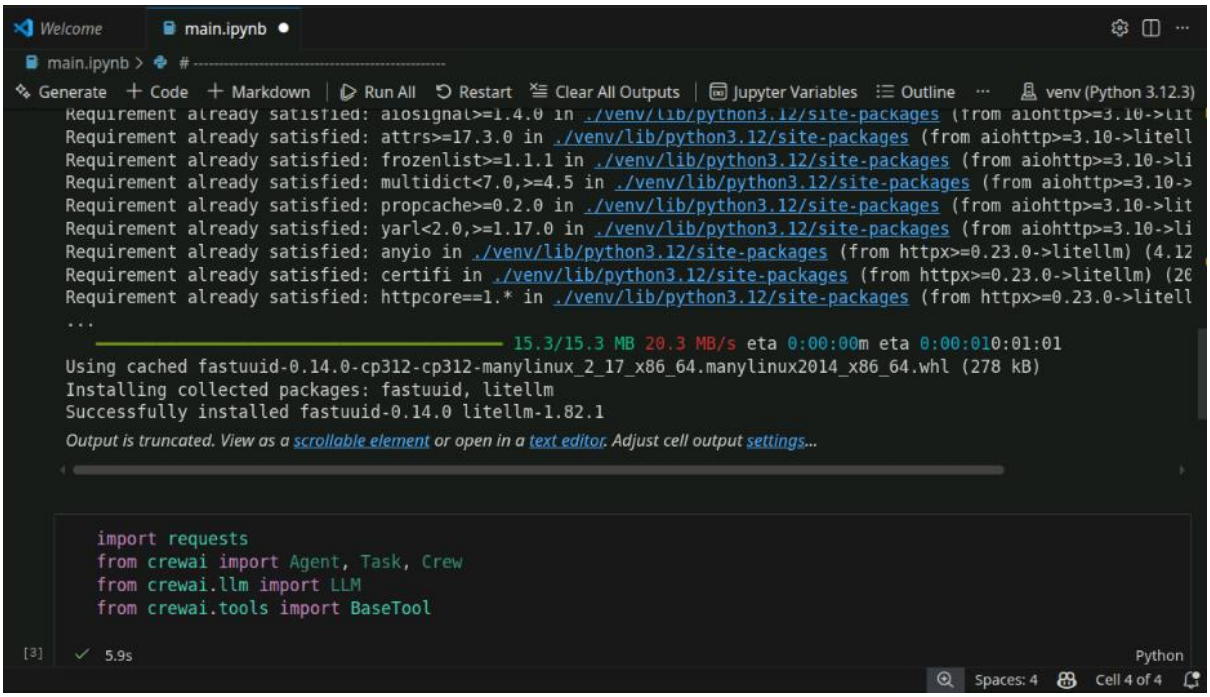
1.4 In the next cell, write the following command to install litellm:

!pip install litellm



1.5 Import the required libraries using the following code:

import requests
from crewai import Agent, Task, Crew
from crewai.llm import LLM
from crewai.tools import BaseTool

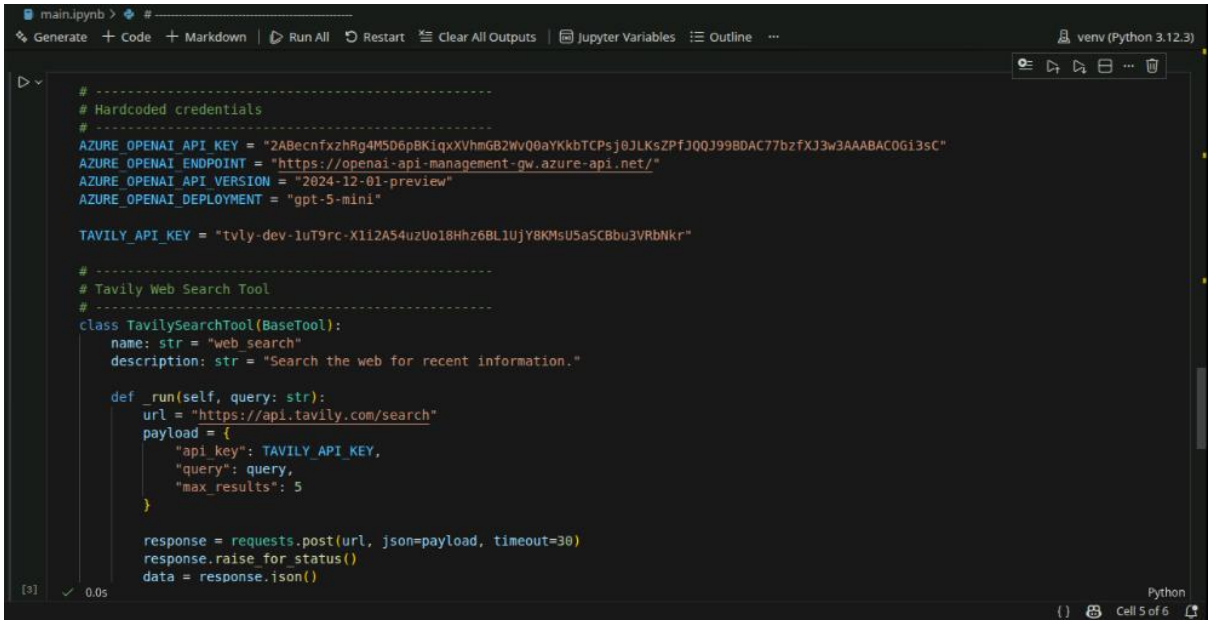


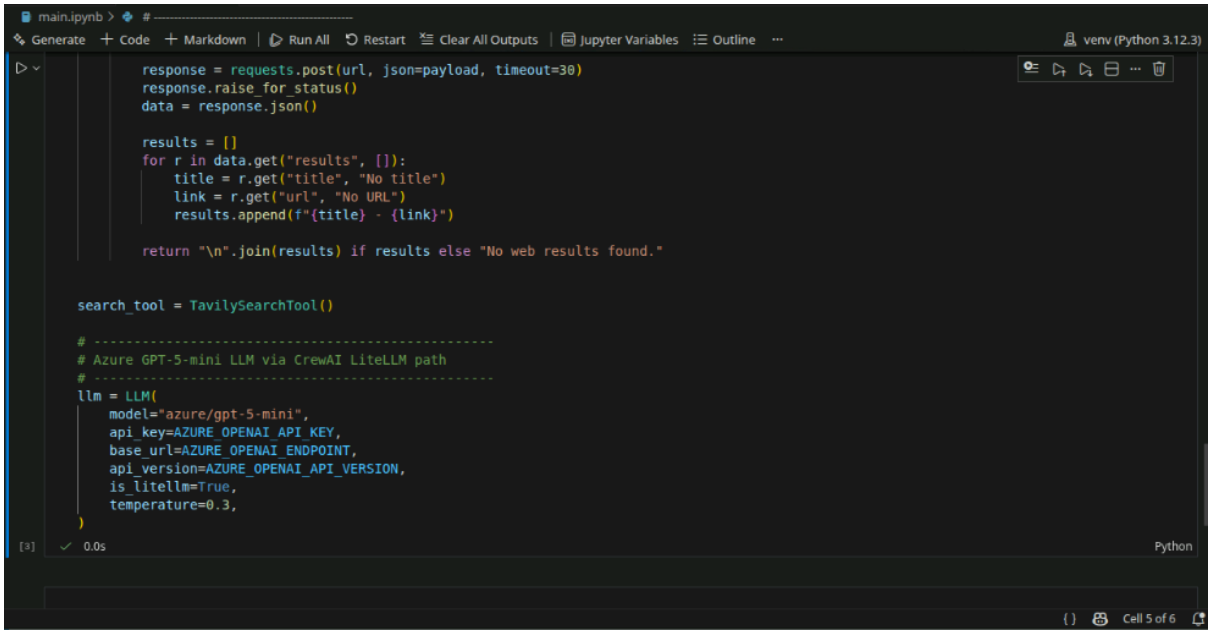
Step 2: Configure the Tavily API Key and Azure OpenAI model

2.1 Use the Tavily Key and Azure OpenAI API key to build an LLM using the following code:

```
# ---------------------------------------------------
# Hardcoded credentials
# ---------------------------------------------------
AZURE_OPENAI_API_KEY =
"2ABecnfxzhRg4M5D6pBKiqxXVhmGB2WvQ0aYKkbTCPsj0JLKsZPfJQQJ99BDAC77b
zfXJ3w3AAABACOGi3sC"
AZURE_OPENAI_ENDPOINT = "https://openai-api-management-gw.azure-api.net/"
AZURE_OPENAI_API_VERSION = "2024-12-01-preview"
AZURE_OPENAI_DEPLOYMENT = "gpt-5-mini"

TAVILY_API_KEY = "tvly-dev-1uT9rc-
X1i2A54uzUo18Hhz6BL1UjY8KMsU5aSCBbu3VRbNkr"

# ---------------------------------------------------
# Tavily Web Search Tool
# ---------------------------------------------------
class TavilySearchTool(BaseTool):
    name: str = "web_search"
    description: str = "Search the web for recent information."

    def _run(self, query: str):
        url = "https://api.tavily.com/search"
        payload = {
            "api_key": TAVILY_API_KEY,
            "query": query,
            "max_results": 5
        }

        response = requests.post(url, json=payload, timeout=30)
        response.raise_for_status()
        data = response.json()

        results = []
        for r in data.get("results", []):
            title = r.get("title", "No title")
            link = r.get("url", "No URL")
            results.append(f"{title} - {link}")

        return "\n".join(results) if results else "No web results found."
```

```
search_tool = TavilySearchTool()

# ----------------------------------------------------
# Azure GPT-5-mini LLM via CrewAI LiteLLM path
# ----------------------------------------------------
llm = LLM(
    model="azure/gpt-5-mini",
    api_key=AZURE_OPENAI_API_KEY,
    base_url=AZURE_OPENAI_ENDPOINT,
    api_version=AZURE_OPENAI_API_VERSION,
    is_litellm=True,
    temperature=0.3,
)
```





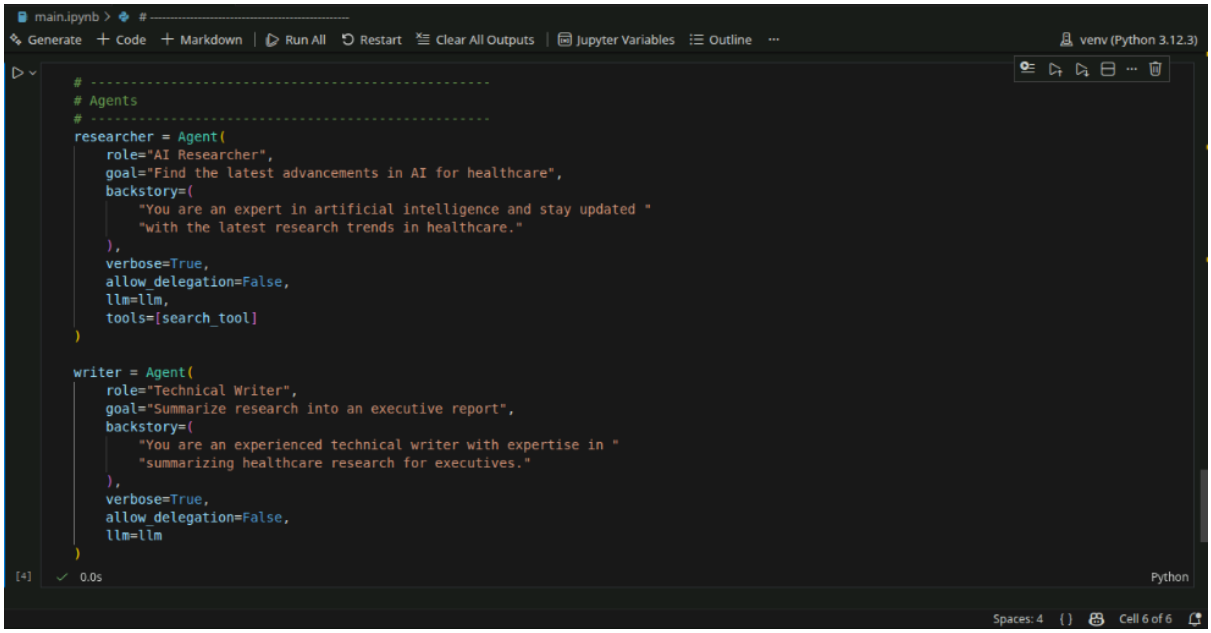
Note: Generate your own Tavily Key and use it in the above code block.

Step 3: Define agents and tasks

3.1 Use the following code to create agents:

```
# ----------------------------------------------------
# Agents
# ----------------------------------------------------
researcher = Agent(
    role="AI Researcher",
    goal="Find the latest advancements in AI for healthcare",
    backstory=(
        "You are an expert in artificial intelligence and stay updated "
        "with the latest research trends in healthcare."
    ),
    verbose=True,
    allow_delegation=False,
    llm=llm,
    tools=[search_tool]
)

writer = Agent(
    role="Technical Writer",
    goal="Summarize research into an executive report",
    backstory=(
        "You are an experienced technical writer with expertise in "
        "summarizing healthcare research for executives."
    ),
```

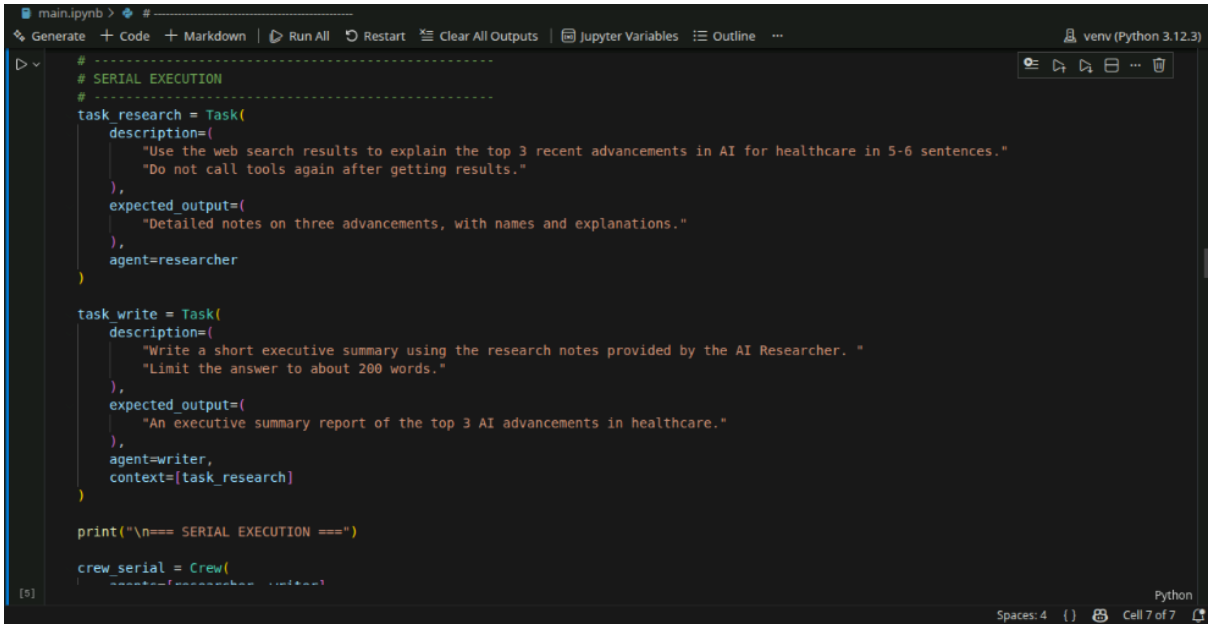
```
    verbose=True,
    allow_delegation=False,
    llm=llm
)
```



3.2 Define and run serial tasks using the following code:

```
# ----------------------------------------------------
# SERIAL EXECUTION
# ----------------------------------------------------
task_research = Task(
    description=(
        "Use the web search results to explain the top 3 recent advancements in AI for healthcare in 5-6 sentences."
        "Do not call tools again after getting results."
    ),
    expected_output=(
        "Detailed notes on three advancements, with names and explanations."
    ),
    agent=researcher
)

task_write = Task(
    description=(
        "Write a short executive summary using the research notes provided by the AI Researcher. "
        "Limit the answer to about 200 words."
    ),
    expected_output=(
        "An executive summary report of the top 3 AI advancements in healthcare."
```

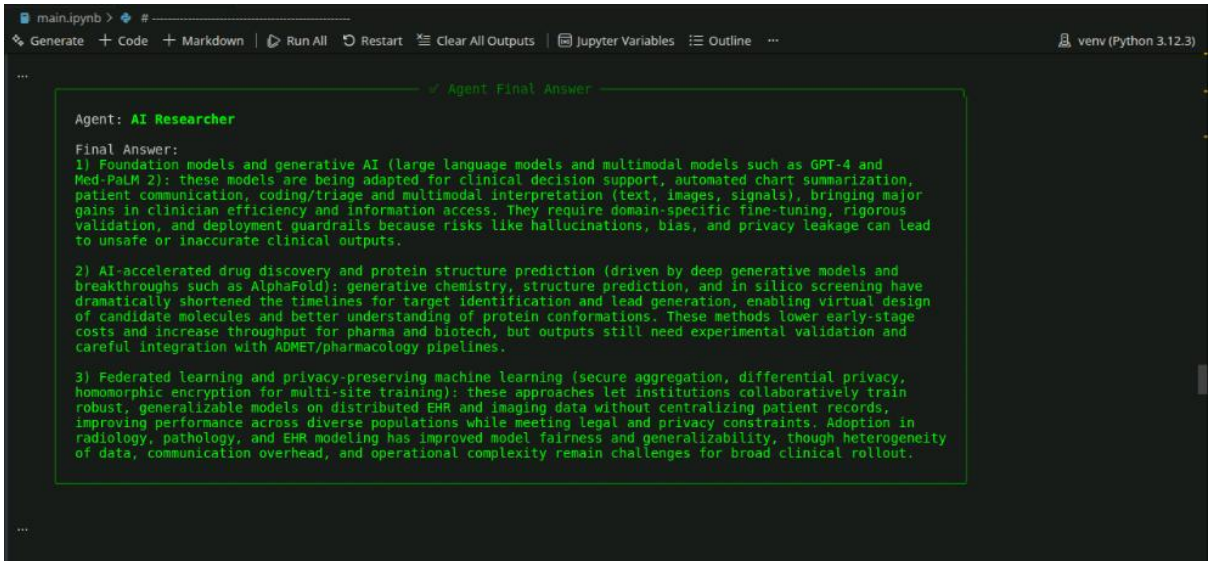
```
    ),
    agent=writer,
    context=[task_research]
)

print("\n=== SERIAL EXECUTION ===")

crew_serial = Crew(
    agents=[researcher, writer],
    tasks=[task_research, task_write],
    verbose=True
)

serial_result = crew_serial.kickoff()

print("\n[Serial Result]:\n")
try:
    print(serial_result.raw)
except AttributeError:
    print(serial_result)
```

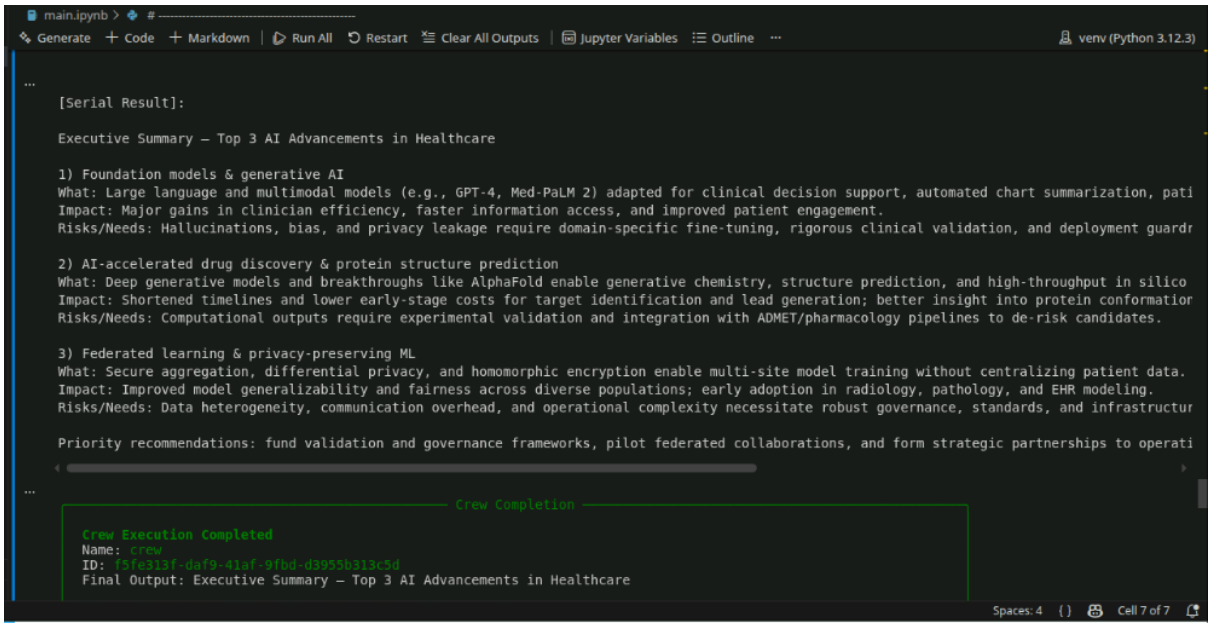


```python
task_write = Task(
    description=(
        "Write a short executive summary using the research notes provided by the AI Researcher. "
        "Limit the answer to about 200 words."
    ),
    expected_output=(
        "An executive summary report of the top 3 AI advancements in healthcare."
    ),
    agent=writer,
    context=[task_research]
)

print("\n=== SERIAL EXECUTION ===")

crew_serial = Crew(
    agents=[researcher, writer],
    tasks=[task_research, task_write],
    verbose=True
)

serial_result = crew_serial.kickoff()

print("\n[Serial Result]:\n")
try:
    print(serial_result.raw)
except AttributeError:
    print(serial_result)
```

The program generates the following output showing serial execution:

```
=== SERIAL EXECUTION ===

Crew Execution Started
Name: crew
ID: f5fe313f-daf9-41af-9fbd-d3955b313c5d

Task Started
Name: Use the web search results to explain the top 3 recent advancements in AI for healthcare in 5-6
sentences.Do not call tools again after getting results.
ID: 70305978-e761-458e-b74e-dfb83e06a233

Agent Started
```





Step 4: Define and run parallel tasks

4.1 Now, define and run the parallel tasks using the following code:

```
# ----------------------------------------------------
# PARALLEL EXECUTION
# ----------------------------------------------------
task_parallel_1 = Task(
    description=(
        "Use web search to list 5 AI companies focusing on drug discovery. "
        "For each company, give one short line about what they specialize in."
    ),
    expected_output="Company names and what they specialize in.",
```

```
    async_execution=True,
    agent=researcher
)

task_parallel_2 = Task(
    description=(
        "Write a short report on how AI is transforming patient diagnostics. "
        "Limit the answer to about 200 words."
    ),
    expected_output="A short report with examples and explanation.",
    agent=writer
)

print("\n=== PARALLEL EXECUTION ===")

crew_parallel = Crew(
    agents=[researcher, writer],
    tasks=[task_parallel_1, task_parallel_2],
    verbose=True
)

parallel_result = crew_parallel.kickoff()

print("\n[Parallel Result]:\n")
try:
    print(parallel_result.raw)
except AttributeError:
    print(parallel_result)
```

main.ipynb > # ----------

Generate | + Code | + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | venv (Python 3.12.3)

```python
# ----------------------------------------------
# PARALLEL EXECUTION
# ----------------------------------------------
task_parallel_1 = Task(
    description=(
        "Use web search to list 5 AI companies focusing on drug discovery. "
        "For each company, give one short line about what they specialize in."
    ),
    expected_output="Company names and what they specialize in.",
    async_execution=True,
    agent=researcher
)

task_parallel_2 = Task(
    description=(
        "Write a short report on how AI is transforming patient diagnostics. "
        "Limit the answer to about 200 words."
    ),
    expected_output="A short report with examples and explanation.",
    agent=writer
)

print("\n=== PARALLEL EXECUTION ===")

crew_parallel = Crew(
    agents=[researcher, writer],
    tasks=[task_parallel_1, task_parallel_2],
    verbose=True
)
```

[6] Python

Spaces: 4 { } Cell 8 of 8

main.ipynb > # ----------

Generate | + Code | + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ... | venv (Python 3.12.3)

```python
    agent=researcher
)

task_parallel_2 = Task(
    description=(
        "Write a short report on how AI is transforming patient diagnostics. "
        "Limit the answer to about 200 words."
    ),
    expected_output="A short report with examples and explanation.",
    agent=writer
)

print("\n=== PARALLEL EXECUTION ===")

crew_parallel = Crew(
    agents=[researcher, writer],
    tasks=[task_parallel_1, task_parallel_2],
    verbose=True
)

parallel_result = crew_parallel.kickoff()

print("\n[Parallel Result]:\n")
try:
    print(parallel_result.raw)
except AttributeError:
    print(parallel_result)
```

[6] Python

The program generates the following output showing parallel execution:

```
main.ipynb > # ----------
Generate  + Code  + Markdown | Run All  Restart  Clear All Outputs | Jupyter Variables  Outline  ...   venv (Python 3.12.3)

=== PARALLEL EXECUTION ===

[] Crew Execution Started

Crew Execution Started
Name: crew
ID: 8ef75636-25d1-4f95-bc8b-12275bdc7967

Task Started

Task Started
Name: Use web search to list 5 AI companies focusing on drug discovery. For each company, give one short line
about what they specialize in.
ID: f5a2b2e4-eb30-4926-b9a6-a3e0e33aef63

[] Agent Started

Agent: AI Researcher
Spaces: 4  { }  Cell 8 of 8
```

```
main.ipynb > # ----------
Generate  + Code  + Markdown | Run All  Restart  Clear All Outputs | Jupyter Variables  Outline  ...   venv (Python 3.12.3)

Tool web_search executed with result: What Does Atomwise Do? AI-Powered Drug Discovery ... - https://www.promptloop.com/directory/what-does-atc
Atomwise - AI Solution Features, Pricing & Reviews - https://welcome.ai/solution/atomw...
Tool web_search executed with result: Exscientia: a clinical pipeline for AI-designed drug candidates - UKRI - https://www.ukri.org/who-we-are/
Tool web_search executed with result: AI-Native Drug Discovery using Insilico Medicine's Nach01 Model ... - https://techcommunity.microsoft.com
Tool web_search executed with result: Recursion: Pioneering AI Drug Discovery - https://www.recursion.com/
How Recursion Pharmaceuticals is Using AI to Revolutionize Drug ... - https://machine-learning-made-simple.medium.com/how-recursion...
Tool web_search executed with result: Artificial intelligence for the discovery of novel antimicrobial agents ... - https://pmc.ncbi.nlm.nih.gc
BenevolentAI: Company Overview by Joanna Shields, CEO - YouTube - https:...

✓ Agent Final Answer

Agent: AI Researcher

Final Answer:
Atomwise: AI-driven structure-based virtual screening and small-molecule lead discovery using deep learning
for docking and binding prediction.
Exscientia: Generative-AI and active-learning platform for designing optimized small-molecule drug candidates
and accelerating lead-to-clinic timelines.
Insilico Medicine: Deep generative models and multi-omics AI for novel molecule design, target
identification, and aging-related biomarker discovery.
Recursion Pharmaceuticals: Image-based phenomics platform that combines high-throughput cellular imaging with
machine learning to discover therapeutics and repurpose compounds.
BenevolentAI: Knowledge-graph and AI-driven target identification and drug repurposing platform focused on
hypothesis generation and translational insights.

Spaces: 4  { }  Cell 8 of 8
```

```
main.ipynb > # ----------
Generate  + Code  + Markdown | Run All  Restart  Clear All Outputs | Jupyter Variables  Outline  ...  venv (Python 3.12.3)

...
                    ✓ Agent Final Answer

Agent: Technical Writer

Final Answer:
AI is reshaping patient diagnostics by enabling earlier, more precise, and scalable detection through pattern
recognition, biomarker discovery, and multimodal data integration.

Examples and mechanisms
- Image-based phenomics (Recursion Pharmaceuticals): High-throughput cellular imaging plus machine learning
extracts disease-specific phenotype signatures, supporting cellular diagnostics, patient stratification, and
rapid assessment of therapeutic response.
- Multi-omics and biomarker discovery (Insilico Medicine): Deep generative models that integrate genomics,
proteomics, and metabolomics accelerate identification of diagnostic biomarkers and signatures for complex
diseases and aging-related conditions.
- Knowledge-graph and literature mining (BenevolentAI): AI links disparate biological data and literature to
surface candidate diagnostic markers and mechanistic hypotheses, shortening the path from signal to clinical
assay.
- Generative chemistry and structure-based platforms (Atomwise, Exscientia): Although focused on drug
discovery, their rapid in-silico screening and design methods enable discovery of molecular probes and
companion diagnostics that validate disease targets and improve diagnostic specificity.

Impact
AI delivers faster discovery of diagnostic signatures, enhanced sensitivity/specificity through multimodal
fusion, and scalable screening for companion diagnostics—supporting personalized care and trial enrichment.

Challenges
Clinical validation, regulatory approval, explainability, data heterogeneity, and integration into clinical
workflows remain critical barriers. Priority actions include prospective validation studies, standardized
pipelines, and clinician-centric implementation to realize operational benefit.

Spaces: 4  {}  Cell 8 of 8
```

```
main.ipynb > # ----------
Generate  + Code  + Markdown | Run All  Restart  Clear All Outputs | Jupyter Variables  Outline  ...  venv (Python 3.12.3)
                    Task Completion

Task Completed
Name: Write a short report on how AI is transforming patient diagnostics. Limit the answer to about 200
words.
Agent: Technical Writer

...

...
[Parallel Result]:

AI is reshaping patient diagnostics by enabling earlier, more precise, and scalable detection through pattern recognition, biomarker discovery,

Examples and mechanisms
- Image-based phenomics (Recursion Pharmaceuticals): High-throughput cellular imaging plus machine learning extracts disease-specific phenotype
- Multi-omics and biomarker discovery (Insilico Medicine): Deep generative models that integrate genomics, proteomics, and metabolomics acceler
- Knowledge-graph and literature mining (BenevolentAI): AI links disparate biological data and literature to surface candidate diagnostic marke
- Generative chemistry and structure-based platforms (Atomwise, Exscientia): Although focused on drug discovery, their rapid in-silico screenir

Impact
AI delivers faster discovery of diagnostic signatures, enhanced sensitivity/specificity through multimodal fusion, and scalable screening for c

Challenges
Clinical validation, regulatory approval, explainability, data heterogeneity, and integration into clinical workflows remain critical barriers.

...
Spaces: 4  {}  Cell 8 of 8
```

By following these steps, you successfully built intelligent CrewAI agent workflows that execute tasks sequentially or in parallel, structuring execution around task interdependencies and concurrency.